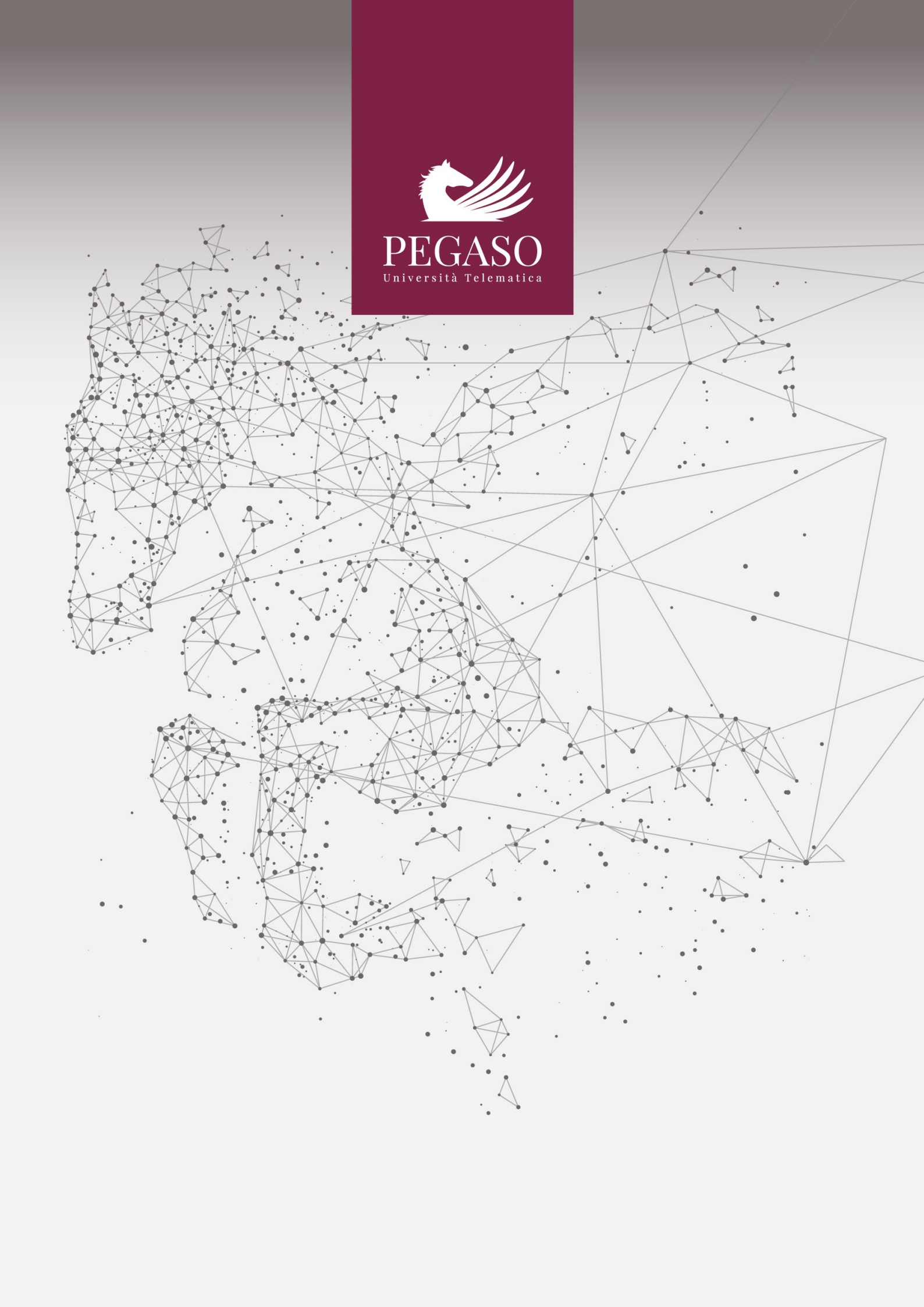

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. ALGORITMI DI BASE 3
2. NETWORK TIME PROTOCOL 6
3. ALGORITMO DI BERKELEY 8
BIBLIOGRAFIA 10

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Algoritmi di base

Se una macchina è equipaggiata con un ricevitore UTC, la sincronizzazione con tutte le altre macchine diventa l'obiettivo da raggiungere. Se nessuna macchina dispone di ricevitori UTC, ogni macchina tiene traccia del proprio tempo e l'obiettivo è quello di mantenere sincronizzate quante più macchine possibile. Nel corso del tempo, sono stati proposti molti algoritmi per ottenere questa sincronizzazione.

Tutti i clock sono basati su un oscillatore armonico, che è un oggetto che risuona a una frequenza specifica e da cui il tempo può essere derivato.

Gli orologi atomici si basano sulle transizioni estremamente rapide e stabili dell'atomo di cesio-133. La maggior parte dei clock hardware dei computer sono basati su oscillatori a cristallo di quarzo, che sono anche in grado di produrre una frequenza molto alta e stabile, anche se non stabile come gli orologi atomici.

Il clock del software di un computer è derivato dal clock hardware del computer. In particolare, si presume che il clock hardware causi un interrupt *f* volte al secondo. Quando questo timer si esaurisce, il gestore di interrupt aggiunge 1 ad un contatore che conta il numero di tic (interrupt) da un tempo predeterminato nel passato. Questo contatore funziona come un clock software *C* che risuona ad una frequenza di *F*.

Si denota con $C_p(t)$ il valore del clock del software sulla macchina *p* quando l'ora UTC è t. L'obiettivo degli algoritmi di sincronizzazione del clock è quello di mantenere la differenza tra i clock di due macchine qualsiasi in un sistema distribuito entro un limite specificato, noto come precisione π:

$$\forall t, \forall p, q: \left| C_p(t) - C_q(t) \right| \leq \pi$$

Si noti che la precisione si riferisce alla differenza di velocità di clock solo tra macchine di sistema distribuite. Quando si parla di un punto di riferimento esterno, come UTC, ci riferiamo alla sua accuratezza, mirando a mantenerlo legato al valore α:

$$\forall t, \forall p: \left| C_p(t) - t \right| \leq \alpha$$

Lo scopo della sincronizzazione del clock è quello di mantenere clock precisi o accurati, noti rispettivamente come sincronizzazione interna o esterna.

Una insieme di clock che sono accurati entro un margine di α, saranno precisi entro un margine di $\pi = 2\alpha$. Tuttavia, la precisione non ci consente di trarre conclusioni sull'accuratezza dei clock.

In un mondo perfetto, $C_p(t)$ sarebbe uguale a t per ogni p e per tutte le t, quindi di conseguenza $\pi = \alpha = 0$. Sfortunatamente, sia i clock hardware che quelli software sono suscettibili al clock drift:

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

poiché la loro frequenza non è perfetta ed è influenzata da fattori esterni come la temperatura, i clock su macchine diverse inizieranno gradualmente a registrare orari diversi. Questo è indicato come clock drift rate, che è la differenza per unità di tempo tra un clock di riferimento perfetto e il clock in uso. Il clock drift rate di un tipico clock hardware basato su quarzo è di circa 10^{-6} secondi al secondo, o circa 31,5 secondi all'anno. Esistono clock hardware per computer con clock drift rate significativamente più bassi.

Le specifiche di un clock hardware includono il sua clock drift rate. Se $F(t)$ rappresenta la frequenza effettiva dell'oscillatore del clock hardware al tempo t e F rappresenta la sua frequenza ideale (costante), allora un clock hardware soddisfa le sue specifiche se t soddisfa le seguenti condizioni:

$$\forall t: (1 - \rho) \leq \frac{F(t)}{F} \leq (1 + \rho)$$

Utilizzando gli interrupt hardware, stiamo accoppiando direttamente il clock del software al clock hardware, riducendo così il clock drift rate. In particolare, abbiamo che

$$C_p(t) = \frac{1}{F}\int_0^t F(t)dt$$

E quindi:

$$\frac{dC_p(t)}{dt} = \frac{F(t)}{F}$$

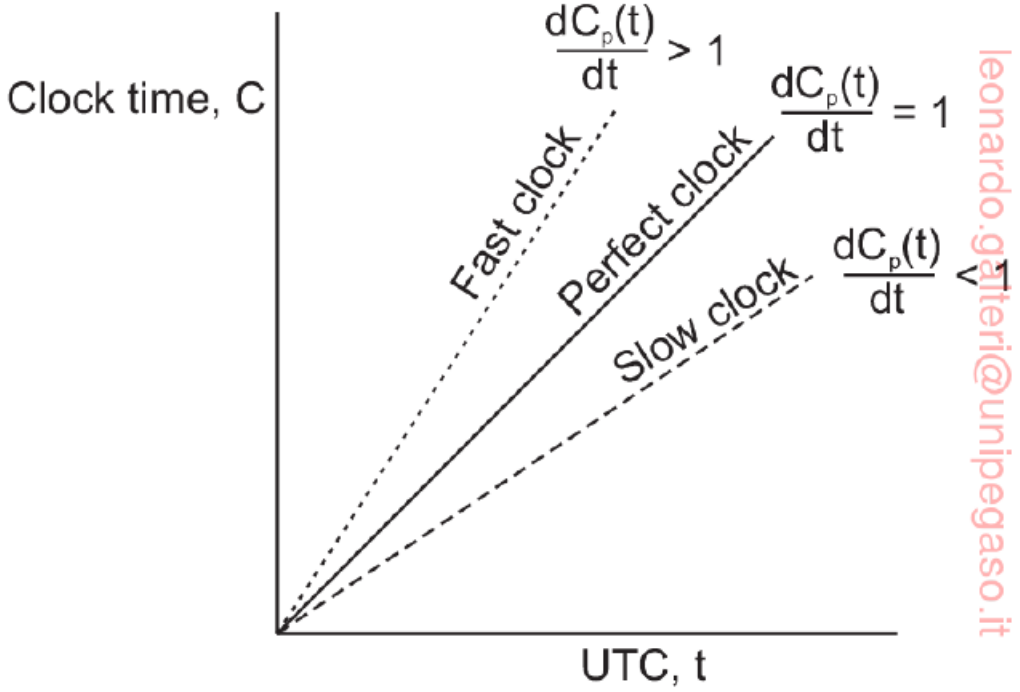
che ci porta al nostro obiettivo finale, che è quello di mantenere il clock drift rate del software limitata a *t*:

$$\forall t: (1 - \rho) \leq \frac{dC_p(t)}{dt} \leq (1 + \rho)$$

La figura 1 illustra orologi lenti, perfetti e veloci.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



leonardo.galteri@unipegaso.it

Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Se due orologi si allontanano dall'UTC in direzioni opposte dopo essere stati sincronizzati, potrebbero essere distanti fino a $2\rho \cdot t$. Gli orologi devono essere risincronizzati (via software) almeno ogni $\frac{\pi}{(2\rho)}$ secondi se i progettisti di sistemi desiderano garantire una precisione di π. Il modo preciso in cui viene eseguita questa risincronizzazione varia tra gli algoritmi.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Network Time Protocol

Un approccio comune in molti protocolli è quello di consentire ai client di comunicare con un time server. Quest'ultimo è dotato di un ricevitore UTC o di un orologio preciso, che consente di fornire con precisione l'ora corrente, ad esempio. Ovviamente, quando si contatta il server, i ritardi dei messaggi avranno reso obsoleto il tempo segnalato. La sfida consiste nel determinare una stima accurata di questi ritardi. Si prenda ad esempio lo scenario illustrato nella Figura 2.

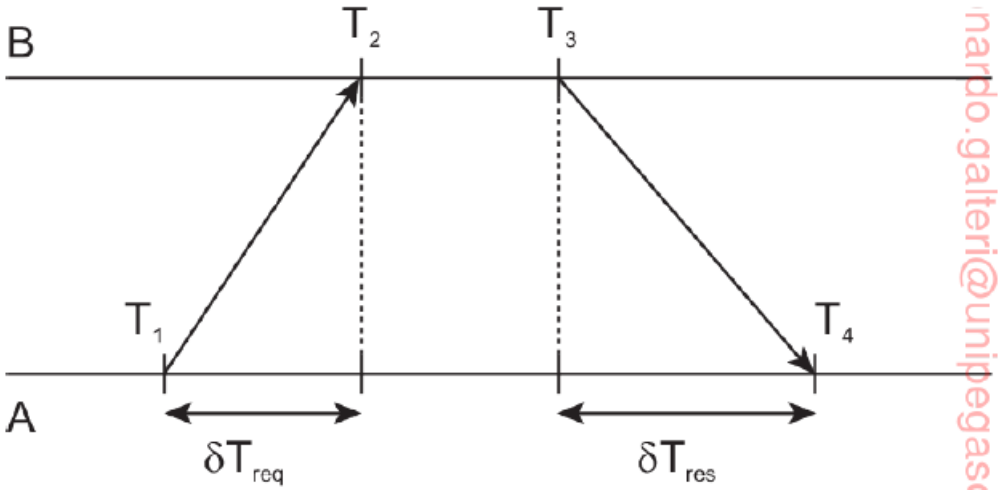


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

In questo scenario, A invierà una richiesta a B con il timestamp T1. B, a sua volta, registrerà l'ora della ricezione T2 (come determinato dal proprio orologio locale) e restituirà una risposta con il timestamp con il valore T3, piggybacking del valore precedentemente registrato T2. Infine, A registra l'ora di arrivo della risposta, T4. Supponendo che i ritardi di propagazione da A a B siano approssimativamente equivalenti a quelli da B ad A, ovvero:

$$\delta T_{req} = T2 - T1 \approx T4 - T3 = \delta T_{res}$$

In tal caso, A può stimare il suo offset rispetto a B come:

$$\theta = T3 + \frac{(T2 - T1) + (T4 - T3)}{2} - T4 = \frac{(T2 - T1) + (T3 - T4)}{2}$$

Ovviamente, il tempo non può tornare indietro. Se il clock di A è veloce, allora si ha che $\theta < 0$, indicando che A dovrebbe impostare in teoria il suo clock all'indietro. In pratica, questa operazione non è

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

consentita perché potrebbe causare gravi problemi, ad esempio un file oggetto compilato subito dopo la modifica dell'ora con un'ora precedente rispetto al file di origine modificato appena prima della modifica del clock.

Nel caso del protocollo NTP (Network Time Protocol), questo protocollo viene stabilito tra coppie di server. In altre parole, anche B si informerà sul tempo di A. L'offset viene calcolato come descritto sopra, insieme alla stima del ritardo:

$$\delta = \frac{(T4 - T1) + (T3 - T2)}{2}$$

Otto coppie di valori (θ, δ) vengono memorizzate nel buffer, con il valore minimo trovato per δ come stima più accurata per il ritardo tra i due server e il valore associato θ che funge da stima più accurata per l'offset.

L'applicazione simmetrica di NTP dovrebbe, in teoria, consentire anche a B di sincronizzare il suo orologio con quello di A. Tuttavia, se l'orologio di B è noto per essere più preciso, sarebbe ingenuo fare una tale regolazione. NTP suddivide i server in *strata* come soluzione a questo problema. Un server con un clock di riferimento, come un ricevitore UTC o un orologio atomico, è noto come server stratum-1 (il clock stesso è indicato come operante allo *stratum* 0). Quando A contatta B, regolerà il suo tempo solo se ha un livello di *stratum* superiore a B. Inoltre, dopo la sincronizzazione, il livello dello strato di A sarà un livello superiore a quello di B. Se B è un server *stratum*-k, allora A diventerà un server di *stratum*-(k + 1) se il suo livello di *stratum* era già maggiore di k. A causa della simmetria di NTP, se A ha un livello di strato inferiore a B, B si adatterà ad A.

NTP ha numerose caratteristiche essenziali, molte delle quali riguardano l'identificazione e il mascheramento degli errori, nonché gli attacchi alla sicurezza. NTP è noto per raggiungere una precisione (globale) compresa tra 1 e 50 millisecondi.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Algoritmo di Berkeley

In numerosi algoritmi di sincronizzazione del clock, il time server è un'entità passiva. Periodicamente, altre macchine lo interrogano per il time corrente, quindi risponde semplicemente alle loro domande. L'approccio esattamente opposto è adottato in Berkeley Unix. In questo caso, il time server (in realtà il demone *ntpd*) è attivo, interrogando periodicamente ogni macchina per determinare che ore sono. Calcola un tempo medio basato sulle risposte e istruisce tutte le altre macchine a regolare i loro orologi alla nuova ora o a rallentare i loro clock fino a raggiungere una riduzione specificata. Questo metodo è appropriato per un sistema in cui nessuna macchina dispone di ricevitori UTC. Periodicamente, l'ora del demone deve essere impostata manualmente dall'operatore. La procedura è illustrata nella Figura 3.

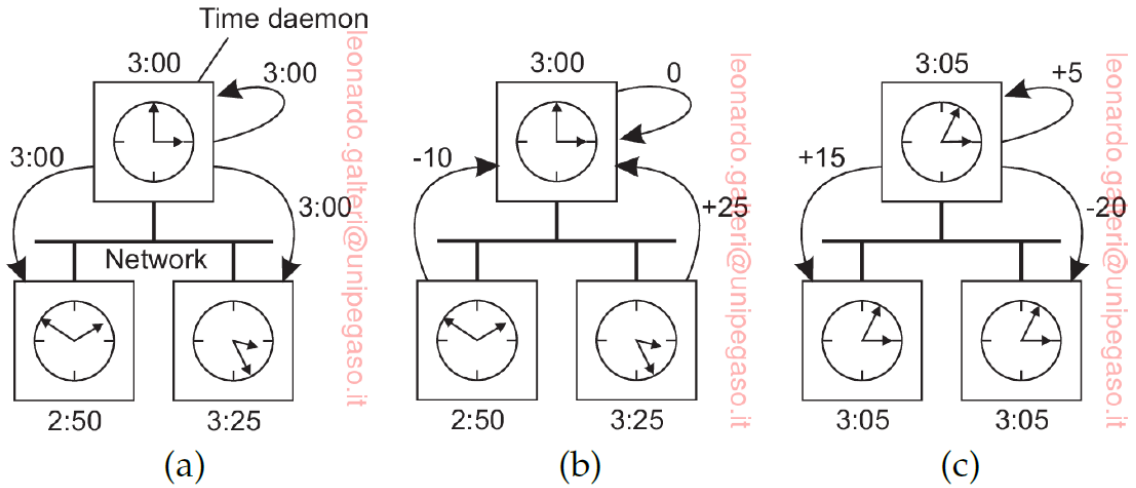


Figura 3 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Alle 3:00 della Figura 3(a), il demone informa le altre macchine del suo tempo e richiede il loro. Nella Figura 3(b), indicano se sono avanti o indietro rispetto al demone. Una volta raccolte queste cifre, il demone calcola la media e istruisce ogni macchina su come regolare il suo clock, come mostrato in Figura 3(c).

In molti contesti, è sufficiente che tutte le macchine condividano lo stesso tempo. Questo tempo non deve necessariamente coincidere con l'ora effettiva trasmessa alla radio ogni ora. Nello scenario illustrato nella Figura 3, se il clock del demone non viene mai calibrato manualmente, non vi è alcun danno finché nessun altro nodo comunica con computer esterni.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Tutti saranno semplicemente d'accordo sul tempo attuale, indipendentemente dalla sua relazione con la realtà. L'algoritmo di Berkeley viene quindi tipicamente utilizzato per sincronizzare un clock interno.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Ramanathan P., Shin K., and Butler R. Fault-Tolerant Clock Synchronization in Distributed Systems. Computer, 1990.
- Horauer M. Clock Synchronization in Distributed Systems. Ph.D., University of Vienna, Department of Computer Science, 2004.
- Shin M., Park M., Oh D., Kim B., and Lee J. Survey on the Clock Synchronization Schemes for Propagation Delay Measurement. International Journal of Advanced Science and Technology, 2011.
- Cristian F. Probabilistic Clock Synchronization. Distributed Computing, 1989.
- Mills D. Network Time Protocol (version 3): Specification, Implementation, and Analysis. RFC 1305, 1992.
- Mills D. L. Computer Network Time Synchronization: The Network Time Protocol on Earth and in Space. CRC Press, 2nd edition, 2011.
- Gusella R. and Zatti S. The Accuracy of the Clock Synchronization Achieved by TEMPO in Berkeley UNIX 4.3BSD. IEEE Transactions on Software Engineering, 1989.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).